



*Software Requirements Specification
Version 1.0*

ApBot

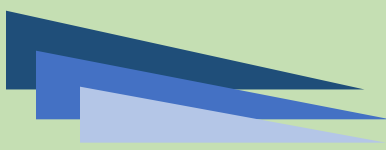
Theme: E-Commerce Assistant

Category: Unleashing the Power of AIML for Intelligent Solutions



Contents

1.1	Background and Necessity for the Application.....	2
1.2	Proposed Solution	3
1.3	Purpose of the Document.....	5
1.4	Scope of Project.....	5
1.5	Constraints.....	6
1.6	Functional Requirements.....	6
1.7	Non-Functional Requirements	8
1.8	Interface Requirements	9
1.8.1	Hardware	9
1.8.2	Software	10
1.9	Project Deliverables.....	10



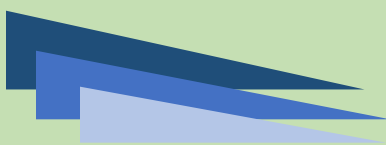
1.1 Background and Necessity for the Application

Many technology-oriented companies are striving to develop an intelligent chatbot that possesses the capability to comprehend, conceptualize, and analyze user communication content. Chatbots are employed across various platforms and industries, not only to enhance communication, but also to optimize the utilization of human resources. By utilizing chatbots, organizations can effectively reduce their expenditure on human resources.

Chatbots significantly enhance the user experience by swiftly resolving routine inquiries. They completely revolutionize customer service encounters. In the market, there are diverse types of chatbots available, each designed for specific purposes and possessing varying capabilities. Support chatbots or quick reply chatbots, for instance, can be found in food delivery apps and e-commerce platforms, enabling prompt solutions for customers and reducing typical waiting times.

Chatbots offer several advantages over traditional customer service channels, such as phone calls and email. Chatbots are available 24/7, can respond quickly to customer inquiries, they are scalable and can be personalized. E-commerce assistant bots can collect data that can be used to improve the customer experience.





1.2 Proposed Solution

One of the key aspects of online shopping is the availability of 24/7 customer service provided by shops or e-commerce Websites. This convenience has been made possible through the utilization of e-commerce assistants also called chatbots. Chatbots are software applications that employ AI to engage in conversations with customers, either through text or speech, to provide assistance during their Website visits. In the past, chatbots were primarily used to offer predefined customer responses. However, thanks to advancements in Artificial Intelligence (AI), it is now feasible to guide potential customers in their purchasing decisions by providing tailored replies based on their preferences and requirements.

The chatbot can address problem areas such as enhancing lead to customer conversion, facilitating product bundling, improving support effectiveness, providing round-the-clock support, and promoting in-person experiences.

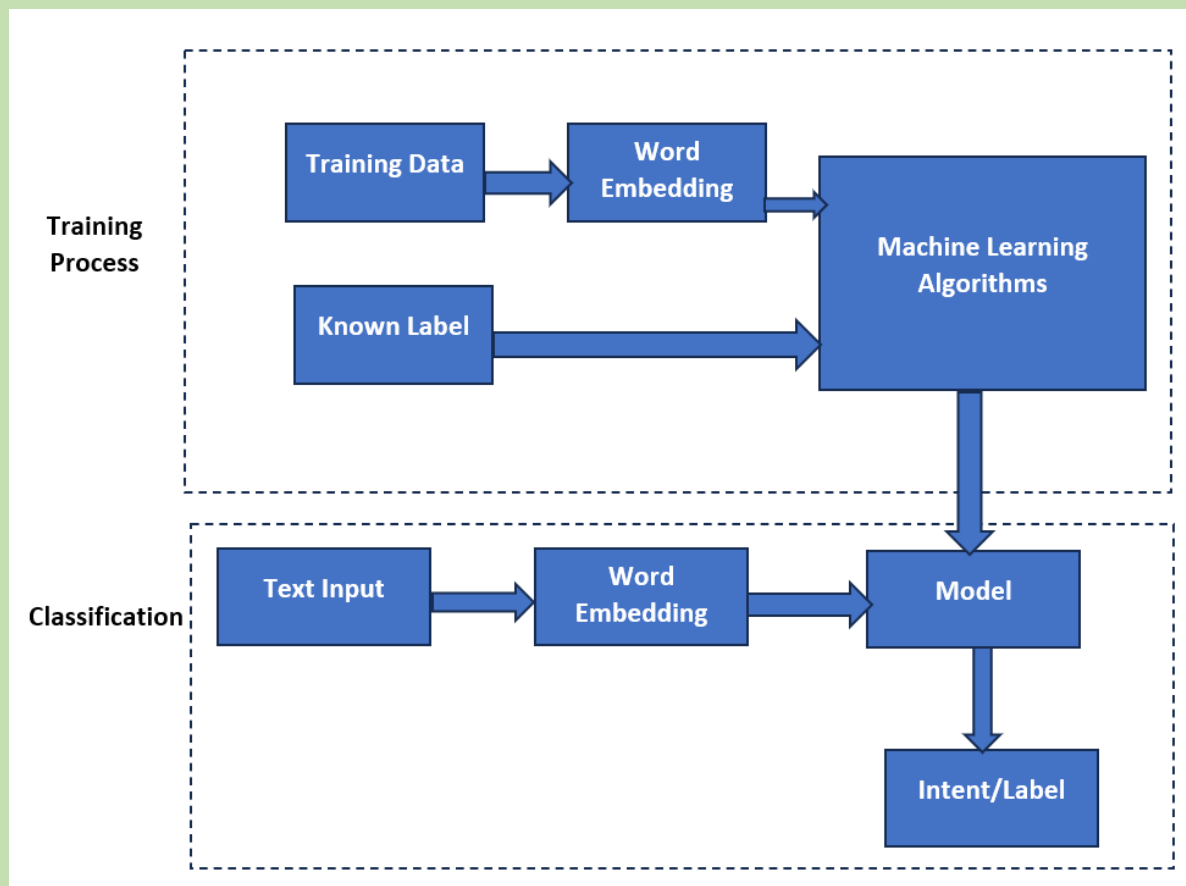
Examples of e-commerce assistant interactions include:

- Welcoming the user upon entering the Website.
- Presenting a range of products and highlighting available offers.
- Providing detailed information about selected products and suggesting similar alternatives from other brands.
- Offering a standard response directing the customer to contact a customer representative via the toll-free number.

The process of building a chatbot involves three main components:

- 1. Creating the Website:** The initial step is to build a simple Website where the chatbot will be deployed. The Website will integrate the chatbot, which is exposed as a Representational State Transfer (REST) API.
- 2. Developing the chatbot:** The chatbot itself is developed using frameworks and APIs like TensorFlow, Keras, and Natural Language Toolkit (NLTK).
- 3. Word vectorization using Bag of Words (BoW):** Within the NLTK framework, words are vectorized using the Bag of Words approach.

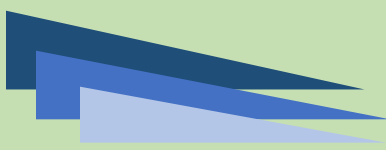
The sample architecture for building the chatbot can be as follows:



Sample Architecture of the Application

The development phase of ApBot includes following steps:

1. Preparing the Natural Language Understanding (NLU) training data
2. Defining the NLU model configuration
3. Compiling and fitting the model
4. Testing the model
5. Integrating the Chatbot with the Website



1.3 Purpose of the Document

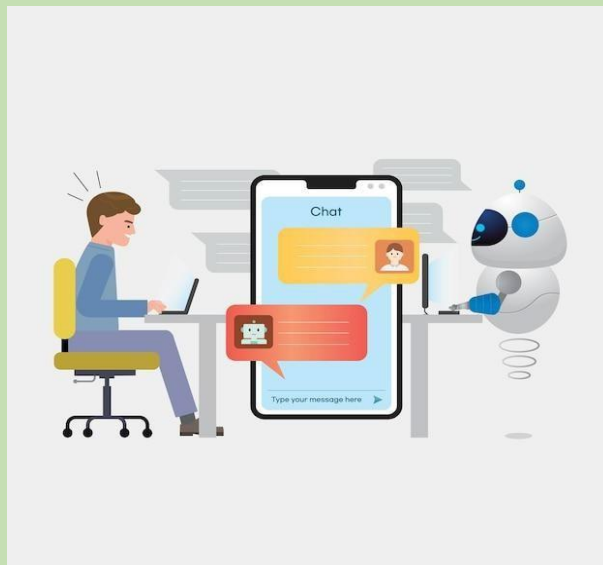
The purpose of this document is to present a detailed description of the AI and ML simulating human conversation by matching user prompts to scripted responses titled '**ApBot**'.

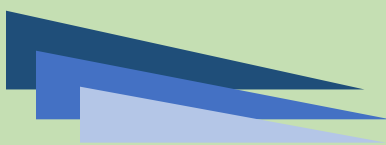
This document explains the purpose and features of the AI and ML simulation of human conversation and the constraints under which it must operate. This document is intended for both stakeholders and developers of the application.

1.4 Scope of Project

The **ApBot** should be able to assist customers by providing information about products, helping with order tracking, addressing frequently asked questions, and resolving common issues.

Chatbots typically seamlessly integrate with various existing applications such as inventory management, order processing, customer databases, and payment gateways. However, integrating with these applications can require significant effort and coordination and is considered beyond the scope of this project.





1.5 Constraints

Chatbots rely on vast amounts of data to understand user inquiries and provide accurate responses. Ensuring the availability and quality of data, by user's way of texting such as product information, customer details, and order history can be challenging. Also, understanding and interpreting user queries accurately can be complex, especially when dealing with natural language variations, slang, or ambiguous requests.

1.6 Functional Requirements

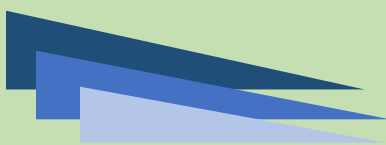
The UI (Website) can be built using any Web framework. The ApBot will be built using Keras and TensorFlow. The chatbot will undergo training using a dataset that includes categories (intents), patterns, and corresponding responses. The basic architecture of the e-commerce assistant will contain the following:

- i. **Pattern Matching** - To ensure effective communication, a list of anticipated inputs and corresponding expected outputs is compiled. This information is then, used to create an intents.json file that encompasses all the identified patterns. Once these patterns are established, the chatbot must comprehend the contextual nuances of the conversation.



- ii. **Natural Language Understanding (NLU)** - Natural Language Understanding (NLU) plays a crucial role in identifying entities, context, and expectations within a chatbot application. Entities refer to specific concepts or ideas that the chatbot requires to recognize, such as greetings, farewells, stores, searches, and more. Context helps determine the background or context of a question, for instance, whether it relates to a previous question or requires the chatbot to track the conversation history to provide a suitable response. Expectations specify the desired outcome or response that is anticipated from the chatbot based on the user's input.

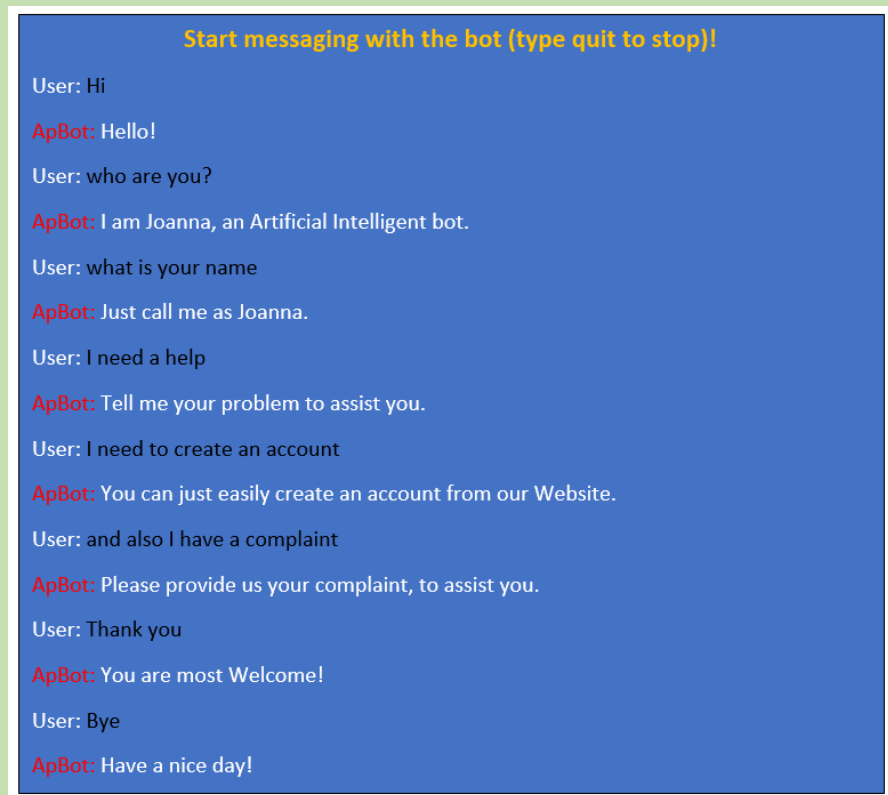




- iii. **Natural Language Processing (NLP)** - Natural Language Processing (NLP) is employed to convert the user's speech or text into a format that can be understood by computers. NLP encompasses various tasks, including tokenization and word vectorization. Tokenization involves breaking down text into individual tokens, such as words or sentences. Word vectorization, on the other hand, represents words as arrays of numerical values to enable computational processing. In the context of NLP, frameworks like Keras and TensorFlow are commonly utilized to facilitate these tasks.

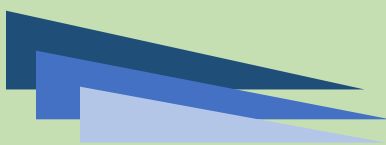


The input to the bot will be in the form of a regular human conversation. Following is an example of expected bot conversation:



The ApBot may be integrated with the main application to:

- Help the user to decide the best possible product.
- Redirect the user to the checkout form if the user decides to make an online purchase.



To construct a chatbot, three key components are involved:

- 1. Frontend:** This component focuses on creating the User Interface (UI) required for the chatbot, enabling users to interact with the application.
- 2. Backend:** It receives and processes data transmitted from the frontend. The backend interacts with a Machine Learning REST API to determine the intent of the user's input. The API returns the identified intent along with its probability. Based on this information, the backend refers to an **intents.json** file to retrieve the appropriate tag and response. Finally, the response is sent back to the frontend for display in the chat UI.
- 3. Machine Learning Model:** The machine learning model is constructed using Keras and TensorFlow. This model is responsible for identifying the intent based on the received data. It utilizes trained algorithms and techniques to make predictions and classify user input according to different intents. By leveraging machine learning, the chatbot can effectively understand and respond to user queries.

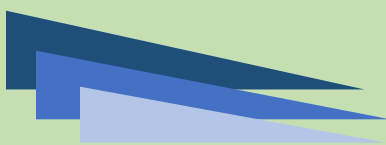


1.7 Non-Functional Requirements

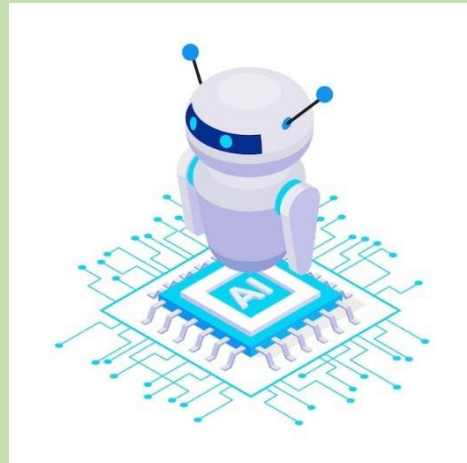
There are several non-functional requirements that should be fulfilled by the Chatbot.

The application should be:

- 1. Compatible:** The proposed application should be designed to ensure compatibility with all Web browsers.
- 2. Secure:** The application should be designed to prevent unauthorized users from gaining access to it.
- 3. User Experience:** The chatbot should provide a seamless and user-friendly experience. It should have an intuitive interface, understand user inputs accurately, and generate relevant and meaningful responses. It should also handle errors gracefully and provide clear instructions or guidance when needed.



4. **Performance:** The chatbot should respond quickly and efficiently to user queries, ensuring minimal latency and smooth interactions. It should be capable of handling multiple concurrent users without compromising its performance.
5. **Reliable:** The chatbot should be reliable and available for users at all times. It should handle errors and exceptions gracefully, recover from failures, and have mechanisms in place to prevent data loss or application crashes.



These are the bare minimum expectations from the project. **It is a must to implement the functional and non-functional requirements given in this SRS.** Once they are complete, you can use your own creativity and imagination to add more features if required.

1.8 Interface Requirements

1.8.1 Hardware

Intel Core i5 Processor or higher
8 GB RAM or higher
Color SVGA
500 GB Hard Disk space
Mouse
Keyboard



1.8.2 Software

Technologies to be used:

1. **Frontend:** HTML5 or any other frontend programming languages
2. **Data Store:** JSON/TXT
3. **Programming/IDE:** Python 3.x, Jupyter Notebook 7.x, Anaconda 3.x or higher, Google Colab
4. **Libraries:** TensorFlow, NLTK, NLU, Keras

1.9 Project Deliverables

You will design and build the project and submit it along with a complete project report that includes:

- Problem Definition
- Design specifications
- Diagrams such as Dialog Flow
- The source code, including. ipynb files for Jupyter notebook and Google Colab, should be shared via GitHub. Appropriate access permissions should be granted to users to allow testing for Jupyter notebook and Google Colab.
- Test Data Used in the Project
- Project Installation Instructions
- The Website code which is integrated with the chatbot
- Link of GitHub for accessing the uploaded project code (Link should have public access)

The consolidated project must be submitted on GitHub with a ReadMe.doc file listing assumptions (if any) made at your end. Ensure that you provide the GitHub URL where the project has been uploaded for sharing. The repository on GitHub should have public access. Documentation is very important part of the project hence, all crucial aspects of the project must be documented properly. Ensure that documentation is complete and comprehensive.

In addition, you must submit a video clip showing the actual working of the application. Over and above the given specifications, you can apply your creativity and logic to improve human conversation.

~~~ End of Document ~~~